

Python knows how to display objects of its built-in types, such as integers and strings. But it can't know how to display objects of the classes we create. If `repr()` doesn't know how to create a repr or str string for an object, by convention the string will be enclosed in angle brackets and contain the object's memory address and class name: '`<wizcoin.WizCoin object at 0x00000212B4148EE0>`'. To create this kind of string for a `WizCoin` object, enter the following into the interactive shell:

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10)
>>> str(purse)
'<wizcoin.WizCoin object at 0x00000212B4148EE0>'
>>> repr(purse)
'<wizcoin.WizCoin object at 0x00000212B4148EE0>'
>>> purse
<wizcoin.WizCoin object at 0x00000212B4148EE0>
```

These strings aren't very readable or useful, so we can tell Python what strings to use by implementing the `__repr__()` and `__str__()` dunder methods. The `__repr__()` method specifies what string Python should return when the object is passed to the `repr()` built-in function, and the `__str__()` method specifies what string Python should return when the object is passed to the `str()` built-in function. Add the following to the end of the `wizcoin.py` file:

```
--snip--
def __repr__(self):
    """Returns a string of an expression that re-creates this object."""
    return f'{self.__class__.__qualname__}({self.galleons}, {self.sickles}, {self.knuts})'

def __str__(self):
    """Returns a human-readable string representation of this object."""
    return f'{self.galleons}g, {self.sickles}s, {self.knuts}k'
```

When we pass `purse` to `repr()` and `str()`, Python calls the `__repr__()` and `__str__()` dunder methods. We don't call the dunder methods in our code.

Note that f-strings that include the object in braces will implicitly call `str()` to get an object's str string. For example, enter the following into the interactive shell: